

UNIVERSIDAD DE GRANADA

Práctica 2

Autores: Yuntao Chen, Adrian Perez Rodriguez

Asignatura: Recuperación de Información

2026-2027

1.Documentos: **3**
2.Inventario de la colección: **4**
2.Contenido, metadatos y enlaces **6**
2.1 Formatos elegidos para la comparación 7
2.2 Calidad del texto extraído 7
2.3 Enlaces 8
2.4 Metadatos útiles para RI 8
2.5 Metadatos poco fiables o dependientes del formato 8
3.Documentos compuestos **9**
3.1 Problemas observados 9
3.2 Procesamiento mediante RecursiveParserWrapper 9
3.3 Estrategia de indexación: 10
4.Representación normalizada 11
4.1 Estado y tratamiento de errores 11
4.2 Limitaciones 11
5. Términos y ley de Zipf 12
5.1 Procesamiento del texto y generación del csv 12
5.2 Gráficos rango-frecuencia, log-log y ley de Zipf 13
5.3 Interpretación de los resultados 15
6. Del documento al buscador 16
6.1 ¿Qué campos conservaría para cada sistema y cuáles indexaría? 16
6.2 ¿Qué metadatos permitirían aplicar filtros durante la recuperación? 18
6.3 ¿Qué consecuencias tendría una extracción incorrecta del texto en cada sistema? 19
6.4 ¿Cómo trataría un documento del que no se ha podido extraer texto? 19
6.5 En el sistema RAG, ¿qué información de procedencia conservaría para poder identificar el documento original a partir de un fragmento recuperado? 20

1.Documentos:

Se ha cogido el siguiente conjunto de documentos:

Documento	**Descripción**
arxiv.pdf	PDf descargado de la página de arxiv sobre un articulo de ondas gravitacional
capturaDePantalla.png	Captura de pantalla del enunciado de la práctica 2
comprimido.zip	Archivo zip incluyendo tres documentos: arxiv.pdf,libro.html y espanol1.txt
Doc1.docx	Archivo docx con una tabla cabecera y pie de página
escaneado.pdf	Tres capturas de pantalla convertido a pdf del enunciado de la práctica 2
espanol1.txt	libro en español del proyecto gutenberg
espanol2.txt	libro en español del proyecto gutenberg
extensionFalse.pdf	libro.html con una extension errónea a propósito
frances.txt	libro en francés del proyecto gutenberg
ingles.txt	libro en inglés del proyecto gutenberg
libro.html	espaniol1.txt en html
P.xlsx	Excel con datos de cursos anteriores
Práctica 1.docx	Práctica 1 en formato docx
prueba.txt	Archivo de prueba de por defecto
wiki.html	html de wikipedia descargada
XML yJSON.pptx	presentación de cursos anteriores en formato pptx

1.1Inventario de la colección:

Se ha generado a partir del anterior conjunto de datos nuestro siguiente inventario:

Fichero	MIME (contenido)	MIME (solo extension)	Codificacion	Idioma	Caracteres	Estado
Doc1.docx	application/vnd.openxmlformats-officedocument.wordprocessingml.document	application/vnd.openxmlformats-officedocument.wordprocessingml.document	-	es	119	OK
GII - Asignaturas y Profesores.xlsx	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	application/vnd.openxmlformats-officedocument.spreadsheetml.sheet	-	es	507032	OK
Práctica 1.docx	application/vnd.openxmlformats-officedocument.wordprocessingml.document	application/vnd.openxmlformats-officedocument.wordprocessingml.document	-	es	20977	OK
XML yJSON.pptx	application/vnd.openxmlformats-officedocument.presentationml.presentation	application/vnd.openxmlformats-officedocument.presentationml.presentation	-	es	5750	OK
arxiv.pdf	application/pdf	application/pdf	-	en	87518	OK
capturaDePantalla.png	image/png	image/png	-	-	0	OK
comprimido.zip	application/zip	application/zip	ISO-8859-1	es	482257	OK
escaneado.pdf	application/pdf	application/pdf	-	-	9	OK
espanol1.txt	text/plain	text/plain	UTF-8	es	1090849	OK
espanol2.txt	text/plain	text/plain	UTF-8	es	217300	OK
extensionFalse.pdf	text/html	application/pdf	UTF-8	es	141888	OK
frances.txt	text/plain	text/plain	UTF-8	fr	317817	OK
ingles.txt	text/plain	text/plain	UTF-8	en	339290	OK
libro.html	text/html	text/html	UTF-8	en	175015	OK
prueba.txt	text/plain	text/plain	UTF-8	es	171	OK
wiki.html	text/html	text/html	UTF-8	es	141888	OK

Observamos como en el apartado de codificación, los archivos .html y .txt siguen el UTF-8 mientras que en los otros como .pptx o .pdf no se detecta una codificación, cabe destacar que el zip tiene una codificación ISO-8859-1, es un aspecto bastante raro ya que dentro incluye un .pdf, .html y .txt, queda en dudas de como tika ha interpretado su codificación.

Por otra parte podemos ver aunque en el archivo de extensionFalse.pdf le hayamos asignado a propósito una extensión errónea (pdf/html), tika ha sabido analizarlo correctamente y sacar todos los datos necesarios.

En escaneado.pdf identifica tan solo 9 caracteres ya que consiste en un pdf con solo tres capturas de imágenes, intuimos que esos 9 caracteres será debido a los saltos de líneas o espacios que hayamos metido, y el arxiv.pdf, tika no ha sido capaz de detectar ningún carácter debido a que son puramente escaneados.

2.Contenido, metadatos y enlaces

Para la extracción contenido de esta parte hemos utilizado para:

- Texto, el **bodyContentHandler** que implementamos en la sección anterior para el recuento de los caracteres, ahora lo único que hemos realizado ha sido almacenarlo en una carpeta *texto* para cada uno de los ficheros disponibles.
- Metadatos, también implementado en la sección anterior por la necesidad del tipo de codificación que tenía el documento, almacena en la carpeta *Metadatos*.
- Idioma, se hace uso de OptimaizeLangDetector con sus modelos para detectar el idioma del documentos mediante los primeros 100.000 caracteres que tiene el documento, si no llega a los 100.000 caracteres se utilizará los que haya, almacenado en la tabla de resumen ya que se exigía en el apartado anterior.
- Enlaces, ha sido el único elemento nuevo que hemos tenido que incluir dado que anteriormente no se pedía, se ha usado el **LinkContentHandler** para la extracción de los enlaces disponibles en los documentos, cabe destacar que como al parser solo se le podía pasar como argumento un único handler, hemos tenido que usar el **TeeContentHandler**, para no tener que hacer la lectura del documento dos veces para la extracción de los links y el texto.

```
-    BodyContentHandler handler = new BodyContentHandler(-1);
-                    LinkContentHandler linkHandler = new
    LinkContentHandler();
-                    TeeContentHandler tee = new
    TeeContentHandler(handler, linkHandler);
-                    try (TikaInputStream tis = TikaInputStream.get(p)) {
-                        parser.parse(tis, tee, md, new ParseContext());
    //el parser actua de locutor y va soltando contenido, mientras el
    tee unifica los dos habndler y cada handler pilla lo que necesite
-                    }
```

Declaramos los dos handler y los “unificamos mediante el TeeContentHandler”, de esta forma con solo un único recorrido habíamos sacado el texto y los enlaces que nos hacía falta.

2.1 Formatos elegidos para la comparación

Hemos escogido wiki.html(HTML),arxiv.pdf(PDF) y Práctica 1 .docx(Docx),para realizar la comparación ya que poseen de una estructura interna distinta.

Aspecto	**HTML**	**PDF**	**DOCX**
Título	Sí	Sí	No
Autor	No	Sí(3 autores)	No aparece en los metadatos pero si en el texto
Fechas	No	No	No
Idioma	es	No especificado	No especificado
Identificadores	No	DOI,arXivID,licencia	no
Nº Metadatos	18	47	3
Enlaces	1700~	150~	120~

2.2 Calidad del texto extraído

En el html observamos que el texto empieza con el menú y muchísimas líneas prácticamente vacías antes de llegar al artículo como tal, el parser al parecer ha extraído todo el cuerpo de la página sin distinguir el contenido de la interfaz.

En el pdf aparentemente el orden de lectura del título, autores y el resumen es correcto, pero nos vamos encontrando con palabras partidas por el guión de fin de línea, números extraños sumados y ruido en la cabecera.

Y en el DOCX, la extracción es bastante buena, el título, los autores y el contenido sale en orden,pero el índice lo ha extraído con los tabuladores también.

2.3 Enlaces

En el HTML la mayoría que observamos son hipervínculos, unas pocas de imágenes y otras que no tienen un tipo asignado.
En el PDF, los enlaces son externos, de direcciones de correo de autores, URL del artículos en arXiv y enlaces a las referencias bibliográficas.
Y en el DOCX no hay ninguno ya que en su momento tampoco se puso por lo que tika tampoco ha sido capaz de sacar ningún contenido.

2.4 Metadatos útiles para RI

Content-type : está en todos los formatos y podríamos usarlo como filtro
dc:title y dc: creator : campos de título y autor que podríamos usar también de filtro aunque solo exista en determinados formatos.
xmpTPg:NPages y pdf:charsPerPage : se puede usar para decir como trocear documentos largos.
Content-language: Nos permite usarlo para el detección del lenguaje.
DOI y arXivID : nos permite identificar y encontrar archivos duplicados rápido.

2.5 Metadatos poco fiables o dependientes del formato

dc:title, og:title,pdf:docinfo:title : hay tres formatos para el titulo, por lo que convendría normalizar para su uso luego.
Directivas de navegador en el HTML como viewport, robots, referrer, format-detection que no aportan nada respecto al contenido.
En el PDF, pikepdf,arXivGenPDF y pdfTex son ejemplos de herramientas que describen la cadena de producción pero no aporta nada en cuanto al RI.

3.Documentos compuestos

Se ha procesado *comprimido.zip*, un contenedor que incluye tres ficheros de la propia colección: espanol2.txt,libro.html y arxiv.pdf, para poder comparar el comportamiento de tika con el contenedor y con los ficheros sueltos.

3.1 Problemas observados

En la parte 1, el parser trato el ZIP como un único documento con 482257 caracteres, idioma es y codificación ISO 8859-1, lo que no describen en absoluto al contenido, que había mezclado los tres ficheros y lo ha tratado como uno único.

3.2 Procesamiento mediante RecursiveParserWrapper

Se ha utilizado el RecursiveParserWrapper para envolver el autoDetectParser que usabamos en la primera parte para que cuando el parser encontrase un recurso embebido que tratase cada uno de los recursos internos por separado , guardando también sus metadatos por separado también, obteniendo de ello la siguiente información:

Elemento	MIME	Idioma	Codificación	Caracteres
el contenedor	applicaction/zip	gl	-	49
espanol2.txt	text/plain	es	UTF-8	217.320
libro.html	text/html	en	UTF-8	175.097
arxi.pdf	application/pdf	en	-	87.678

Observamos que tika identifica el contenedor y sus tres recursos internos, el contenedor no tiene apenas texto propio, el contenido útil está todo en sus recursos, observamos también que se le ha asignado el idioma a cada uno de los ficheros por separado, pero que al contenedor se le ha asignado gl de gallego, que desde luego no es muy fiable ya que eran pocos caracteres lo que podía usar el detector de lenguaje y ha mostrado el resultado que más se ha asemejado a uno de sus modelos.

3.3 Estrategia de indexación:

Estrategia	Ventajas	Inconvenientes
Un único documento	Muy simple	Metadatos confusos, resultado que no indica exactamente de qué fichero proviene el fuente, el texto de uno se diluye a otros
Documentos independientes	Cada recurso con sus metadatos	Se pierde que son del mismo contenedor
Independientes relacionados con el contenedor	Cada recurso con sus metadatos y una ruta vinculado al contenedor	Tendría que guardar un campo adicional para la relación

La más adecuada que consideramos es la de documentos independientes relacionados con el contenedor.

Escenario de Búsqueda: Si por ejemplo tenemos una biblioteca digital donde recibe los documentos agrupados en formato zip, y el usuario busca “gravitional waves” que es de lo que va el artículo arxiv.pdf, el usuario obtendría arxiv.pdf,con sus metadatos únicamente y la ruta , de esta forma podría abrir el contenedor de este y recuperar también el resto del lote.

Si hubiésemos tratado el zip como un único documento entonces el buscador solo puede devolver comprimido.zip sin indicar exactamente cual es el documento relevante, y si hubiese sido como documento independiente, tendríamos el documento correcto pero se pierde la referencia al resto del lote.

4.Representación normalizada

El programa genera la coleccion.jsonl con un objeto JSON por línea y un registro por documento, hemos utilizado la biblioteca de jackson para dicha generación, el texto extraído contiene comillas, saltos de línea y caracteres especiales, y jackson es capaz de escaparlas, razón por la que escogimos su uso.

Los datos que no existen se escriben como null, y no como cadena vacía, para poder permitir distinguir entre lo que no existe y lo que existe pero está vacío.

4.1 Estado y tratamiento de errores

El campo status puede tomar 3 posibles valores:

- OK: Procesado con texto extraído
- SIN_TEXTO: Procesado pero sin texto
- Error <Exception> : Procesamiento fallido

SIN_TEXTO se introdujo porque sin este no podríamos distinguir uno con texto de uno sin texto para poder realizar procesamientos posteriores si quisiéramos.

4.2 Limitaciones

El contenedor se guarda como un único registro, su texto mezcla de los tres ficheros, no se aplica en el JSONL la estrategia elegida en la Parte 3 aunque ya demostramos que era posible, debido a que el contenido de esos tres ficheros queda duplicado por lo que lo dejamos como un único documento.

Además los textos se guardan sin limpiar, podemos ver como en el JSONL se observan los saltos /r /n y la cabecera de licencia en inglés de los ficheros del Proyecto de GutenBerg, esto seguramente afecte a la tokenización y el análisis de frecuencias de la parte 5.

5. Términos y ley de Zipf

Se ha analizado la frecuencia de términos del texto extraído del documento espanol2.txt. Elegimos este documento porque tiene un buen tamaño suficiente para poder tener una amplia variedad de términos detectados junto con sus apariciones y porque presenta distintos idiomas (español, francés, inglés) para poder medir las probabilidades de un idioma.

5.1 Procesamiento del texto y generación del csv

Leemos el contenido del fichero usando FIles.readString especificándole la codificación UTF-8. Una vez lo tenemos lo convertimos a minúsculas para que las apariciones de una misma palabra no dependan de si aparece en minúsculas o mayúsculas, así se contabilizan de forma conjunta.

Para identificar los términos hemos usado la expresión regular (\\p{L}+) que reconoce secuencias de una o más letras de Unicode. De esta forma un token será una secuencia de una o más letras. Igualmente en próximas prácticas se podrá implementar Lucene y mejorar la tokenización haciendo uso de StandardTokenizer, por ejemplo.

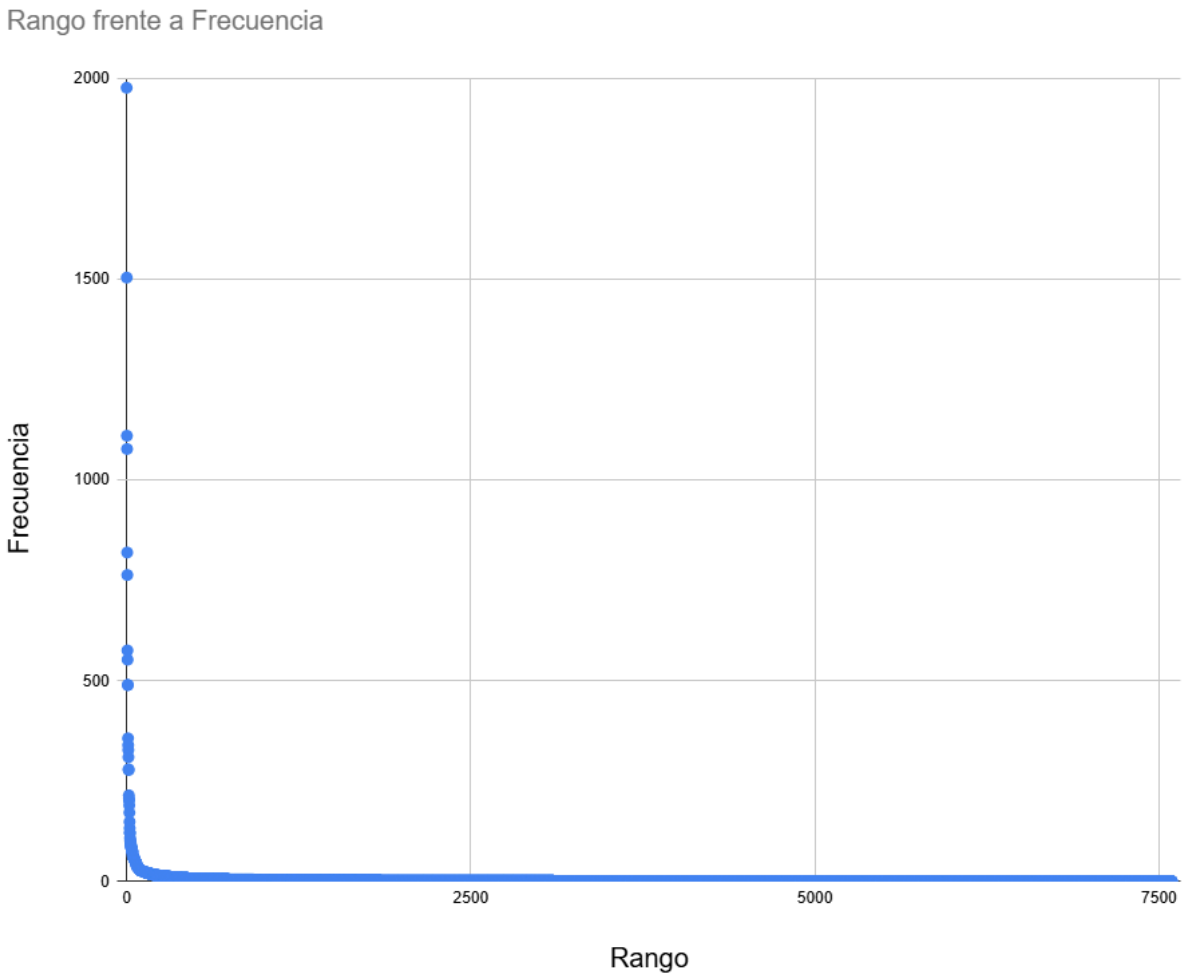
A continuación usaremos un HashMap que almacene los términos junto con sus frecuencias.

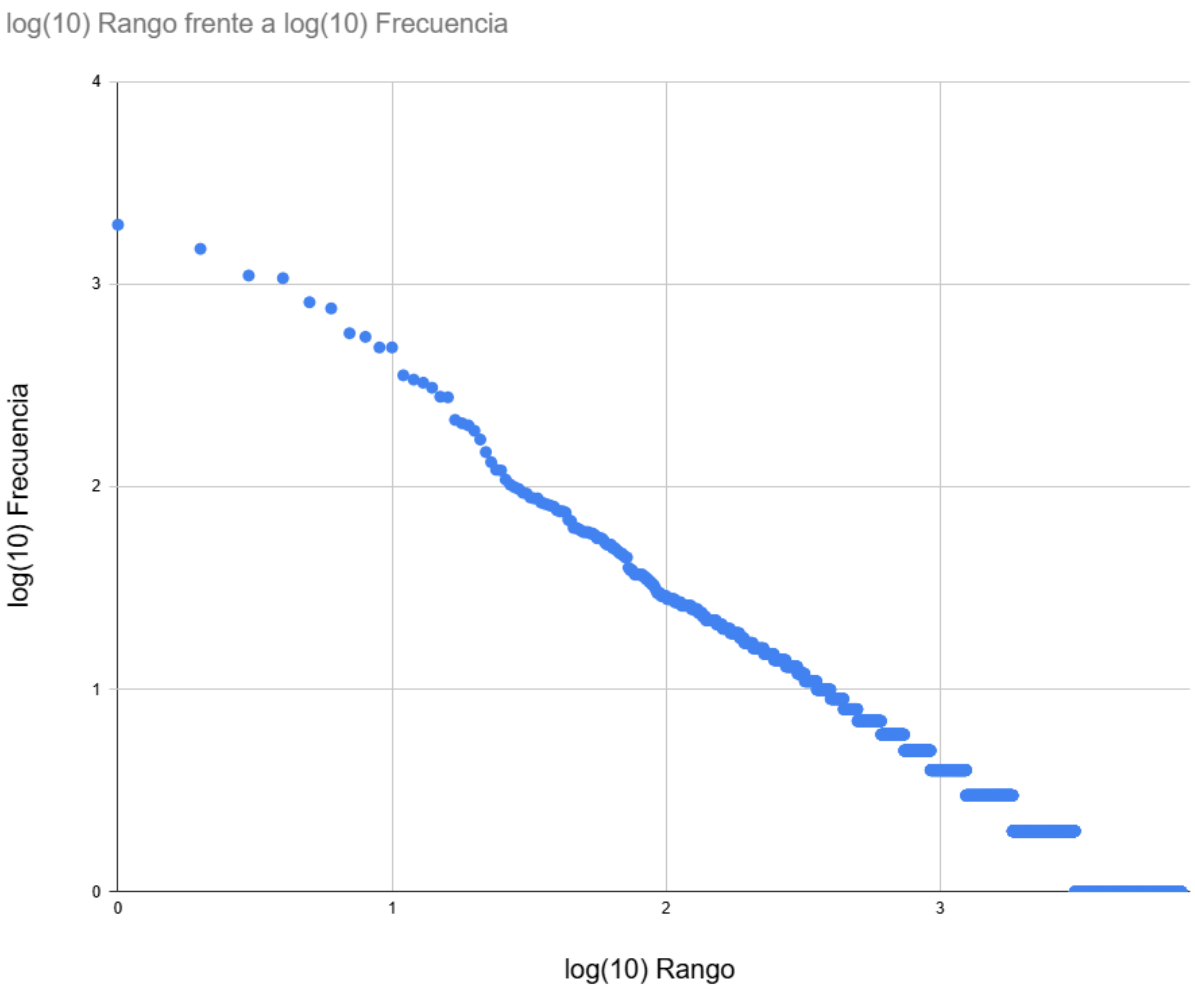
Cuando ya hemos contado los términos, los ordenamos por frecuencia descendente. Si dos términos empatan en apariciones, se ordenan alfabéticamente.

Al final generamos un fichero llamado frecuencias.csv que está formado por dos columnas: término y frecuencia.

Artículos, preposiciones y conjunciones son más frecuentes entre las palabras de su mismo idioma. Palabras como ‘de’, ‘y’ y ‘el’ ocupan los primeros puestos en español. En inglés palabras como ‘the’, ‘of’ y ‘to’. En francés palabras francesas como ‘le’ y otras palabras también usadas en español: ‘la’ (esto afecta a que aparezca aún más veces, pues se suman las apariciones en ambos idiomas)

5.2 Gráficos rango-frecuencia, log-log y ley de Zipf





Veamos si la distribución presenta aproximadamente el comportamiento descrito por la ley de Zipf: F(R) ≈ k / Rm.

Al aplicar logaritmos la convertimos en una ecuación lineal: log F = log k − m log R. Así calculamos la regresión lineal y obtenemos k y m.

La pendiente m la calculamos en la misma hoja de cálculo

G2 =PENDIENTE(D2:D7595;E2:E7595)

	A	B	C	D	E	F	G
1	Termino	Frecuencia	Rango	log(10) Frecuencia	log(10) Rango		Pendiente
2	de	1978	1	3,296226287	0		-0,8485796592
3	y	1505	2	3,1775365	0,3010299957		

Como la fórmula de una recta es y = a + bx, la pendiente obtenida es +b que corresponde con -m (b = -m y m = -b). Por tanto m = 0,8485796592.

Ahora vamos a calcular el intercepto en y de la recta (log k) para obtener k (k = 10^a).

G5 =INTERSECCION.EJE(E2:E7595;D2:D7595)

	A	B	C	D	E	F	G
1	Termino	Frecuencia	Rango	log(10) Frecuencia	log(10) Rango		Pendiente
2	de	1978	1	3,296226287	0		-0,8485796592
3	y	1505	2	3,1775365	0,3010299957		
4	el	1111	3	3,045714059	0,4771212547		Intercepto
5	la	1078	4	3,032618761	0,6020599913		3,715862116
6	en	820	5	2,913813852	0,6989700043		

Intercepto = 3,715862116 → k = 10^3,715862116 → k = 5198.30929395

Obtenemos la función de ley de Zipf: $F(R) \approx \frac{5198{,}31}{R^{0{,}84858}}$

Ahora vamos a calcular la regresión (R^2) para ver si el gráfico log-log presenta una tendencia aproximadamente lineal, como habría que esperar de la ley de Zipf.

G8 =COEFICIENTE.R2(E2:E7595;D2:D7595)

	A	B	C	D	E	F	G
1	Termino	Frecuencia	Rango	log(10) Frecuencia	log(10) Rango		Pendiente
2	de	1978	1	3,296226287	0		-0,8485796592
3	y	1505	2	3,1775365	0,3010299957		
4	el	1111	3	3,045714059	0,4771212547		Intercepto
5	la	1078	4	3,032618761	0,6020599913		3,715862116
6	en	820	5	2,913813852	0,6989700043		
7	que	764	6	2,883093359	0,7781512504		Coeficiente R2
8	los	576	7	2,760422483	0,84509804		0,9553044746

Observamos un coeficiente muy cercano a 1, por lo que el gráfico log-log se ajusta bastante bien a una tendencia lineal como habría que esperar de Zipf.

5.3 Interpretación de los resultados

Como hemos mencionado, un coeficiente R2 tan cercano a 1 indica que existe una relación lineal bastante marcada entre el logaritmo del rango y el logaritmo de la frecuencia, así que los resultados son compatibles con un comportamiento aproximadamente similar al descrito por la ley de Zipf.

También observamos mucha desigualdad en las frecuencias: pocos términos aparecen muchas veces y muchos términos pocas veces. 4492 términos de los 7594 aparecen solo 1 vez, más de la

mitad. La pendiente m = 0,8486 también indica que la frecuencia disminuye al aumentar el rango.

Esto es importante de tener en cuenta para un sistema de recuperación de información, pues no todos los términos aportan información relevante para localizar documentos. Palabras que aparecen mucho como ‘de’ o ‘y’ no ayudan a diferenciar un documento de otro, mientras que términos más específicos sí sirven para identificar temas concretos.
A la hora de construir un índice estas palabras frecuentes se pueden descartar pues no son significantes para identificar documentos.

6. Del documento al buscador

6.1 ¿Qué campos conservaría para cada sistema y cuáles indexaría?

a) Buscador tradicional con índice invertido

En este caso, el campo principal es text que se debe indexar mediante un índice invertido, así cada término apunta a los documentos donde aparece.
Ahora consideremos todos los campos:

Campo	Conservar	Indexar
id	Sí	Sí, como identificador
path	Sí	No, salvo búsqueda por ruta
mime_type	Sí	Sí, como campo filtrable
language	Sí	Sí, como campo filtrable
title	Sí	Sí, índice invertido
author	Sí	Sí, como campo filtrable
text	Sí	Sí, índice invertido
num_characters	Sí	Opcional
links	Sí	Opcional
status	Sí	Sí, como campo filtrable

Title y text se indexan como índice invertido para si se formula una consulta que incluya una palabra del título del documento o de su texto se recupere el documento.
Id es un índice de identificador. Identifica de forma única al documento.
Mime_type, language, author, status los podemos usar para filtrar documentos por estos campos, pues son los más interesantes a la hora de filtrar.
El resto no se indexan pues no aportan información suficientemente relevante a la hora de cribar los documentos.

Por otra parte todos los los campos se conservan porque forman parte del registro JSONL y pueden ser necesarios para mostrar resultados, depurar el proceso, reconstruir el documento o aplicar filtros futuros, pero no todos necesitan formar parte de la estructura de búsqueda.

b) Sistema RAG con embeddings

En un sistema RAG el campo principal sigue siendo text pero no se indexa como un documento completo, el texto se divide en chunks, y cada fragmento se transforma en un embedding. Estos embeddings se almacenan en un índice vectorial, de modo que ante una consulta se pueden recuperar los fragmentos semánticamente más cercanos, aunque no contengan exactamente las mismas palabras que la consulta.

A diferencia del buscador tradicional, en RAG es muy importante conservar los metadatos del documento original en cada fragmento. Esto nos permite identificar la procedencia del fragmento recuperado, mostrar la fuente en la respuesta, aplicar filtros durante la recuperación y reconstruir el contexto original si hace falta.

Campo	Conservar en cada fragmento	Indexar
id (del doc)	Sí	Sí, como identificador
chunk_id	Sí	Sí, como identificador
chunk_index	Sí	Opcional
path	Sí, a nivel de doc	No, salvo búsqueda por ruta
mime_type	Sí, a nivel de doc	Sí, como campo filtrable
language	Sí, a nivel de doc	Sí, como campo filtrable

title	Sí,a nivel de doc	Sí, índice invertido
author	Sí,a nivel de doc	Sí, como campo filtrable
chunk de text	Sí	Sí, con su embedding
embedding	Sí	Sí, ïndice vectorial
num_characters	Sí	Opcional
links	Sí, a nivel de doc	Opcional
status	Sí	Sí, como campo filtrable

A parte se podrían guardar los campos chunk_id y chunk_index para identificar de forma única el fragmento y la posición del fragmento respecto al documento. También, en este caso, no se guarda el text como tal si no el chunk correspondiente y su embedding al cual hace referencia. Este embedding se indexa con un índice vectorial.

Los campos que se guardan a nivel de documento se guardan una sola vez en el propio documento y el chunk se relaciona con estos ya que tiene una referencia al documento que pertenece el chunk.

6.2 ¿Qué metadatos permitirían aplicar filtros durante la recuperación?

Considerando para ambos sistemas:

Como ya hemos visto en el apartado anterior hay varios campos que consideramos como los más útiles para filtrar por ellos. Los elegidos para filtrar han sido mime_type, language, author y status

Otros campos como título o text pueden ser inconvenientes para filtrar y son mapas aptos para buscar contenido

Path no lo indexamos como campo filtrable pues no lo consideramos suficientemente relevante, aunque podría usarse para buscar por ruta determinada si se precisase. Lo mismo aplica para num_characters y links.

Id permite seleccionar un documento concreto por su identificador.

6.3 ¿Qué consecuencias tendría una extracción incorrecta del texto en cada sistema?

a) Buscador tradicional con índice invertido

Si el texto se extrae de forma incompleta o contiene errores, el índice invertido no contendrá correctamente todos los términos del documento. Como consecuencia, algunas búsquedas no recuperarían documentos relevantes porque las palabras buscadas podrían haberse perdido o reconocido incorrectamente. Además, si se extrae texto que no pertenece al contenido principal, como encabezados repetidos o información irrelevante, podrían aparecer resultados poco útiles.

b) Sistema RAG con embeddings

En RAG, una extracción incorrecta también afectaría a la división del texto en fragmentos y a los embeddings generados a partir de ellos. Si falta información o el texto está alterado, los fragmentos podrían perder parte de su contexto y no representar correctamente el contenido original. Esto dificultaría recuperar la información adecuada y podría provocar que el sistema generase respuestas incompletas o incorrectas.

En conclusión, en ambos sistemas es importante realizar una extracción de texto de calidad. En el buscador tradicional los errores perjudican principalmente la recuperación basada en palabras y en RAG pueden afectar a la representación semántica de los fragmentos y, por tanto, a las respuestas generadas.

6.4 ¿Cómo trataría un documento del que no se ha podido extraer texto?

a) Buscador tradicional con índice invertido

El documento no se incorporaría al índice invertido de contenido, ya que no hay términos que indexar. Sin embargo sí que conservamos sus metadatos para poder identificarlo y consultar información sobre él. También se podría intentar procesarlo de nuevo utilizando otro método de extracción si fuera necesario.

b) Sistema RAG con embeddings

No se generarían fragmentos ni embeddings si no se ha obtenido texto útil, ya que no habría contenido que representar semánticamente. El documento y sus metadatos se conservarían, pero se excluiría de la recuperación basada en embeddings hasta que se pudiera extraer su contenido correctamente.

En conclusión, conservamos estos documentos para no perder información sobre la colección, registramos el resultado de la extracción en status y los excluimos de las búsquedas de contenido mientras no tengamos un texto válido.

6.5 En el sistema RAG, ¿qué información de procedencia conservaría para poder identificar el documento original a partir de un fragmento recuperado?

Como hemos comentado en el apartado 6.1, conservamos el id del documento al que hace referencia un chunk para identificar a qué documento pertenece, y una vez recuperamos el documento podemos usar el campo path para recuperar su ruta.
Obtenemos el documento a partir del id, y el path a partir del documento.

Trabajo realizado por cada uno

Adrian Perez Rodriguez	Punto 5,6
Yuntao Chen	Punto 1,2,3,4